

Attribute	iOS	iPadOS	macOS	watchOS	visionOS
UI framework	SwiftUI, UIKit	SwiftUI, UIKit	SwiftUI, AppKit	SwiftUI, WatchKit (WKApplicationDelegateAdaptor)	SwiftUI, RealityKit
Scene types	scene-based life cycle; WindowGroup	UIWindowScene / UISceneSession; WindowGroup(id:for:)	WindowGroup, Window, UtilityWindow, Settings, MenuBarExtra	single-target watch app: dependent or independent	WindowGroup, volume, ImmersiveSpace
Primary input	touch	touch, pointer, Apple Pencil	pointer, keyboard, trackpad	Digital Crown, double tap	look and tap (gaze privacy)
Windowing	one scene per app instance	resizable windows via UIApplicationSupportsMultipleScenes, menu bar	sizing, placement, style, level and restoration per scene; menu bar extra	TabView with verticalPage, NavigationStack	Shared Space (windows, volumes); Full Space (ImmersiveSpace)
Widget surfaces	Home Screen, Lock Screen, StandBy, CarPlay, Notification Center	Home Screen, Today view, StandBy, CarPlay, Notification Center	Mac desktop and Notification Center	complications: watch faces and Smart Stack (accessoryCorner, ...)	widgets on the wall (systemExtraLargePortrait, ...)
Background	BGAppRefreshTask, BGProcessingTask, UIBackgroundModes	BGAppRefreshTask, BGProcessingTask	App Nap; no BackgroundTasks framework	WKExtendedRuntimeSession; WKApplicationRefreshBackgroundTask	BackgroundTask.appRefresh(_:)
Distribution	App Store (App Store Connect, TestFlight)	App Store (App Store Connect, TestFlight)	Mac App Store, or Developer ID signed and notarized	App Store from the watch (independent) or with the iPhone app	App Store: native, or "Designed for iPad" offered automatically
Xcode / SDK	Xcode 27, 27 SDK, deployment minimum 26	Xcode 27, 27 SDK, deployment minimum 26	Xcode 27; Apple silicon only	Xcode 27, 27 SDK, deployment minimum 26	Xcode 27, 27 SDK, deployment minimum 26

IOS: SCENES, LIVE ACTIVITIES, DYNAMIC ISLAND

- Scene phases:** `ScenePhase` `.active` / `.inactive` / `.background`; `.background` is the last reliable moment to save.

```
@Environment(\.scenePhase) var scenePhase
.onChange(of: scenePhase) { _, newPhase in ... }
```
- UIKit:** `UIApplicationDelegate` per process, `UIWindowSceneDelegate` per scene. **27 SDK:** adopt the scene-based life cycle or the app fails to launch: add `UIApplicationSceneManifest`; `applicationDidEnterBackground(_)` → `sceneDidEnterBackground(_)`.
- Live Activities** (`ActivityKit`): `ActivityAttributes` (static) + `ContentState` (dynamic, < 4 KB total); `NSSupportsLiveActivities` = YES; ≤ 8 h, then up to 4 h more on the Lock Screen.

```
let a = try Activity.request(
  attributes: attrs, content: content,
  pushType: .token)
await a.update(ActivityContent(
  state: s, stateDate: d))
await a.end(a.content,
  dismissalPolicy: .immediate)
```
- Dynamic Island:** in `ActivityConfiguration(for:)`, `dynamicIsland`: returns `DynamicIsland` { `DynamicIslandExpandedRegion(.leading)` ... } `compactLeading`: {} `compactTrailing`: {} `minimal`: {} ; check `isDynamicIslandLimitedInWidth`.

MACOS: SCENES, SANDBOX, DISTRIBUTION

- Scenes:** `WindowGroup` (any number), `Window` (exactly one), `UtilityWindow` (floating panel), `Settings` { } opened by `SettingsLink` / `openSettings`, `MenuBarExtra`.

```
MenuBarExtra("FieldNotes",
  systemImage: "note.text") { ... }
.menuBarExtraStyle(.menu) // or .window
.commands {
  CommandGroup(after: .importExport) {
    Button(...).keyboardShortcut("e",
      modifiers: [.command, .shift]) }
  CommandGroup(replacing: .help) { ... }
} // + CommandMenu
```
- App Sandbox:** required on the Mac App Store; `com.apple.security.files.user-selected.read-write`; keep folder access with a security-scoped bookmark (`bookmarkData`, `startAccessingSecurityScopedResource`).
- Developer ID:** Developer ID Application + hardened runtime + secure timestamp; then notarize, staple (`alttool` is retired):

```
xcrun notarytool submit \
  build/FieldNotes.dmg \
  --keychain-profile fieldnotes-notary \
  --wait
xcrun stapler staple \
  build/FieldNotes.dmg
```
- Mac Catalyst** = UIKit iPad app recompiled (`targetEnvironment(macCatalyst)`). **Designed for iPad** = unmodified iPad binary, Apple silicon only; run-time check `isiOSSAppOnMac`.

VISIONOS: SPACES, VOLUMES, REALITYKIT

- Shared Space** (default): windows and volumes of several apps side by side; no camera, room mesh or hand data. **Full Space:** an `ImmersiveSpace`; other apps' windows hidden, ARKit data flows. One immersive space at a time across all apps.

```
WindowGroup(id: "field-site") {
  FieldSiteVolume() }
.windowStyle(.volumetric)
.defaultSize(width: 0.6, height: 0.35,
  depth: 0.6, in: .meters)
ImmersiveSpace(id: "map") {
  NotesMapSpace() }
.immersionStyle(
  selection: mapImmersion, in: .mixed,
  progressive(0.2...1.0,
    initialAmount: 0.5))
```
- Immersion styles:** `.mixed` (room via passthrough), `.progressive` (Digital Crown sets the replaced portion), `.full` (passthrough off).
- `await openImmersiveSpace(id:)` → `.opened` / `.userCancelled` / `.error`; `dismissImmersiveSpace()`.
- `RealityView` { `content`, `attachments` in `content.add(site)` } `update`: { `content`, `attachments` in ... } `attachments`: { `Attachment(id: "note-card")` { ... } }
- `Model3D` for decoration; gestures on entities: `targetedToAnyEntity()`.
- Ornament:** `.ornament(attachmentAnchor: .scene(.bottom))` { `NoteOrnament()` } + `glassBackgroundEffect()`.
- Hover effects:** gaze privacy — the system draws the highlight, the app only learns the tap. `hoverEffect(.highlight)` / `.lift`; entities: `HoverEffectComponent` + `InputTargetComponent` + `CollisionComponent`.

IPADOS: WINDOWS, POINTER, PENCIL, SIDEBAR

- Multiple windows:** `UIApplicationSupportsMultipleScenes` = true in `UIApplicationSceneManifest`; `UIRequiresFullScreen` is deprecated since iPadOS 26. `UISceneClosuresConfirmation` (new in 27) confirms a close.

```
WindowGroup(id: "note",
  for: Note.ID.self) { $noteID in ... }
openWindow(id: "note", value: note.id)
// brings it forward if already open
```
- Pointer:** `hoverEffect(.highlight)` / `.lift` / `.automatic`, shaped by `contentShape(.hoverEffect, _)`. `pointerStyle(_)` is **not** on iPadOS; UIKit: `UIPointerInteraction` → `UIPointerStyle`.
- Apple Pencil:** `PencilKit` `PKCanvasView`; `onPencilDoubleTap`, `onPencilSqueeze` (Pro) with `PencilHoverPose`; `honorPreferredPencilDoubleTapAction` / `preferredPencilSqueezeAction`. UIKit: `UIPencilInteraction`.
- Sidebar:** `TabView` + `.tabViewStyle(.sidebarAdaptable)` — top tab bar ↔ sidebar with `TabSection`; persist `TabViewCustomization` in `@AppStorage`. UIKit: `UITabBarController` `mode` = `.tabSidebar`.

WATCHOS: TARGET, CONNECTIVITY, WORKOUTS

- Single target:** one `@main` SwiftUI App, no WatchKit extension; `WKCompanionAppBundleIdentifier`; independent: `WKRunsIndependentlyOfCompanionApp`.
- `WCSession.default`: delegate, activate early; property-list types only.

<code>sendMessage(_:replyHandler:errorHandler:)</code>	now, only while reachable; else error
<code>transferUserInfo(_:)</code>	queued; every item, in order
<code>updateApplicationContext(_:)</code>	queued; only the latest is kept
<code>transferFile(_:metadata:)</code>	queued file + metadata
- Workouts:** `HKWorkoutSession` + `HKLiveWorkoutBuilder` (workout-processing mode; mirror to iPhone: `startMirroringToCompanionDevice()`).
- Smart Stack relevance:** `TimelineEntryRelevance(score:duration:)`, the provider's `relevance()` or `RelevanceConfiguration` (watchOS 26). iPhone Live Activities appear there too.
- Crown / double tap:** `.focusable()` + `.digitalCrownRotation($minutes, from:through:by:...)`; double tap: `.handGestureShortcut(.primaryAction)`.

CONDITIONAL COMPILATION & AVAILABILITY

Pattern	Evaluated	FieldNotes example
<code>#if os(macOS)</code>	Compile time	Settings and MenuBarExtra scenes
<code>if #available(iOS 27, macOS 27, watchOS 27, visionOS 27, *)</code>	Run time	on-device summary button (new in 27)
<code>if #unavailable(iOS 27, ...)</code>	Run time	one-time pin migration on 26
<code>@available(iOS 27, ..., *)</code>	Compile time (call sites)	SummaryCard; callers must be guarded
<code>@available(watchOS, unavailable)</code>	Compile time	<code>exportNotebook(_)</code> on Apple Watch

- No `iPadOS` condition for `os()`: iPad builds use the iOS SDK. A false `#if` branch is never type-checked.